

Prolog Lab Programs

Artificial Intelligence Laboratory | All 10 Experiments

Lab 1: Factorial of a Number

```
% Base case
factorial(0, 1) :- !.

% Recursive case
factorial(N, F) :-
    N > 0,
    N1 is N - 1,
    factorial(N1, F1),
    F is N * F1.

% Query example:
% ?- factorial(5, F). % F = 120
```

Note: Query: factorial(N, F) where N is the number.

Lab 2: Length of a List

```
% Base case: empty list has length 0
list_length([], 0).

% Recursive case
list_length(_|T, L) :-
    list_length(T, L1),
    L is L1 + 1.

% Query example:
% ?- list_length([a,b,c,d], L). % L = 4
```

Note: Query: list_length([...], L) where L is the resulting length.

Lab 3: Reverse a List

```
% Helper with accumulator
rev([], Acc, Acc).
rev([H|T], Acc, R) :-
    rev(T, [H|Acc], R).

% Main predicate
reverse_list(L, R) :- rev(L, [], R).
```

```
% Query example:
% ?- reverse_list([1,2,3], R). % R = [3,2,1]
```

Note: Query: reverse_list([...], R) where R is the reversed list.

Lab 4: N-Queens Problem

```
queens(N, Qs) :-
    numlist(1, N, Ns),
    permutation(Ns, Qs),
    safe(Qs).

safe([]).
safe([Q|Qs]) :- no_attack(Q, Qs, 1), safe(Qs).

no_attack(_, [], _).
no_attack(Q, [Q1|Qs], D) :-
    Q1 - Q =\= D, Q - Q1 =\= D,
    D1 is D + 1,
    no_attack(Q, Qs, D1).

% Query example:
% ?- queens(4, Qs). % Qs = [2,4,1,3]
```

Note: Query: queens(N, Qs) — Qs is a list of column positions per row.

Lab 5: Depth First Search (DFS) & Breadth First Search (BFS)

```
% Graph edges
edge(a, b). edge(a, c). edge(b, d).
edge(c, d). edge(d, e).

% --- DFS ---
dfs(Goal, Goal, _, [Goal]).
dfs(Start, Goal, Visited, [Start|Path]) :-
    edge(Start, Next),
    \+ member(Next, Visited),
    dfs(Next, Goal, [Next|Visited], Path).

% --- BFS ---
bfs(Start, Goal, Path) :-
    bfs_queue([[Start]], Goal, RevPath),
    reverse(RevPath, Path).

bfs_queue([[Goal|Rest]|_], Goal, [Goal|Rest]).
bfs_queue([[H|T]|Queue], Goal, Path) :-
    findall([Next,H|T],
        (edge(H, Next), \+ member(Next, [H|T])),
        NewPaths),
```

```

append(Queue, NewPaths, NewQueue),
bfs_queue(NewQueue, Goal, Path).

```

```

% Queries:
% ?- dfs(a, e, [a], Path).
% ?- bfs(a, e, Path).

```

Note: DFS uses recursion with visited tracking; BFS uses a queue of paths.

Lab 6: Best First Search

```

% Graph: edge(From, To, Cost)
edge(a, b, 2). edge(a, c, 4).
edge(b, d, 3). edge(c, d, 1). edge(d, e, 2).

% Heuristic values
h(a, 6). h(b, 4). h(c, 3). h(d, 2). h(e, 0).

best_first(Start, Goal, Path) :-
h(Start, H),
best_first_search([[H, Start]], Goal, RevPath),
reverse(RevPath, Path).

best_first_search([[_ , Goal|Rest]|_], Goal, [Goal|Rest]).
best_first_search([[_ ,H|Visited]|Open], Goal, Path) :-
findall([NH, Next, H|Visited],
(edge(H, Next, _), \+ member(Next, [H|Visited])),
h(Next, NH)),
Children),
append(Open, Children, Open1),
sort(Open1, Sorted),
best_first_search(Sorted, Goal, Path).

% Query:
% ?- best_first(a, e, Path).

```

Note: Uses heuristic $h/2$ to guide search. Nodes with lower h values expand first.

Lab 7: 8-Puzzle using Best First Search

```

% State: list of 9 tiles, 0 = blank
goal([1,2,3,4,5,6,7,8,0]).

% Manhattan distance heuristic
heuristic(State, H) :-
goal(Goal),
findall(D, (
nth0(I, State, T), T \= 0,
nth0(J, Goal, T),

```

```

Row1 is I // 3, Col1 is I mod 3,
Row2 is J // 3, Col2 is J mod 3,
D is abs(Row1-Row2) + abs(Col1-Col2)
), Ds),
sumlist(Ds, H).

% Blank moves: up, down, left, right
move(State, Next) :-
nth0(I, State, 0),
( M is I - 3 ; M is I + 3 ;
(M is I - 1, I mod 3 \= 0) ;
(M is I + 1, M mod 3 \= 0) ),
M >= 0, M < 9,
nth0(M, State, T),
copy_term(State, Next),
nth0(I, Next, T), nth0(M, Next, 0).

solve(State, [State]) :- goal(State).
solve(State, [State|Path]) :-
move(State, Next),
solve(Next, Path).

% Query:
% ?- solve([1,2,3,4,0,5,7,8,6], Path), length(Path, L).

```

Note: Use a simple/near-solved start state for fast results. Full Best-First requires priority queue.

Lab 8: Water Jug Problem (Production Rules)

```

% State: state(X, Y) – X=4L jug, Y=3L jug
goal(state(2, _)).

% Production Rules
rule(state(_, Y), state(4, Y), 'Fill 4L jug').
rule(state(X, _), state(X, 3), 'Fill 3L jug').
rule(state(_, Y), state(0, Y), 'Empty 4L jug').
rule(state(X, _), state(X, 0), 'Empty 3L jug').
rule(state(X, Y), state(X1, Y1), 'Pour 4L->3L') :-
T is min(X, 3 - Y), X1 is X - T, Y1 is Y + T.
rule(state(X, Y), state(X1, Y1), 'Pour 3L->4L') :-
T is min(Y, 4 - X), X1 is X + T, Y1 is Y - T.

solve(State, _, [State]) :- goal(State).
solve(State, Visited, [State|Path]) :-
rule(State, Next, _),
\+ member(Next, Visited),
solve(Next, [Next|Visited], Path).

```

```
water_jug(Path) :- solve(state(0,0), [state(0,0)], Path).
```

```
% Query:
```

```
% ?- water_jug(Path).
```

Note: Start state is state(0,0). Goal is to get 2L in the 4L jug.

Lab 9: Monkey Banana Problem

```
% State: state(MonkeyPos, BoxPos, MonkeyOnBox, HasBanana)
```

```
% Banana is at position 'middle', box starts at 'corner'
```

```
initial(state(door, corner, off, no)).
```

```
goal(state(_, _, _, yes)).
```

```
% Actions
```

```
move(state(P,P,off,H), walk(P,Q), state(Q,P,off,H)) :- P\=Q.
```

```
move(state(P,P,off,H), push(P,Q), state(Q,Q,off,H)) :- P\=Q.
```

```
move(state(P,P,off,H), climb, state(P,P,on,H)).
```

```
move(state(middle,middle,on,no), grab, state(middle,middle,on,yes)).
```

```
solve(State, []) :- goal(State).
```

```
solve(State, [Action|Actions]) :-
```

```
move(State, Action, Next),
```

```
solve(Next, Actions).
```

```
% Query:
```

```
% ?- initial(S), solve(S, Actions).
```

Note: Monkey must walk to box, push it under banana, climb, then grab.

Lab 10: A* Search

```
% Graph: arc(From, To, Cost)
```

```
arc(a,b,1). arc(a,c,4). arc(b,c,2).
```

```
arc(b,d,5). arc(c,d,1). arc(d,e,3).
```

```
% Heuristic h(Node, Goal, HValue)
```

```
h(a,e,7). h(b,e,6). h(c,e,2). h(d,e,1). h(e,e,0).
```

```
% astar(+Open, +Goal, -Path)
```

```
% Open = list of f(F,G,Node,Path)
```

```
astar([f(_,_,Goal,P)|_], Goal, Path) :-
```

```
reverse(P, Path).
```

```
astar([f(_,G,Node,Visited)|Open], Goal, Path) :-
```

```
findall(f(F1,G1,Next,[Next|Visited]),
```

```
(arc(Node,Next,C),
```

```
\+ member(Next, Visited),
```

```
G1 is G + C,
```

```
h(Next, Goal, H),
F1 is G1 + H),
Children),
append(Open, Children, Open1),
sort(Open1, Sorted),
astar(Sorted, Goal, Path).

a_star(Start, Goal, Path) :-
h(Start, Goal, H),
astar([f(H,0,Start,[Start])], Goal, Path).

% Query:
% ?- a_star(a, e, Path).
```

Note: $f = g + h$ where g =actual cost, h =heuristic estimate. Expands lowest f -cost node first.

How to Run Prolog Programs

All programs in this lab use **SWI-Prolog**, which is the standard Prolog interpreter for academic and research use. The steps below apply uniformly to all 10 labs.

Step 1 — Install SWI-Prolog

Download from <https://www.swi-prolog.org/download/stable> and install for your OS (Windows, macOS, or Linux). On Ubuntu/Debian: `sudo apt install swi-prolog`

Step 2 — Save the Program

Copy the code into a file, e.g. `lab1.pl`. Use any text editor (VS Code, Notepad++, gedit).

Step 3 — Open SWI-Prolog

Launch the terminal / command prompt and run: `swipl`

Step 4 — Load the File

Inside the Prolog interpreter, type: `?- consult('lab1.pl').`

Or use the short form: `?- [lab1].` (without the `.pl` extension)

Step 5 — Run the Query

Type the query shown in the comment at the bottom of each program, e.g.: `?- factorial(5, F).`

Press Enter. Prolog will display the result. Press `;` for more solutions or `.` to stop.

Step 6 — Exit

Type `halt.` to exit the interpreter.

Labs with different considerations:

- Lab 7 (8-Puzzle): Use a near-solved starting state to keep execution fast, e.g. `[1, 2, 3, 4, 0, 5, 7, 8, 6]`. Randomly shuffled states may cause very slow or infinite execution without a priority queue.
- Lab 9 (Monkey-Banana): The query uses the built-in `initial/1` predicate defined in the file — don't hardcode the state manually in the query.

All other labs (1–6, 8, 10) follow the exact same load-and-query workflow described above.